



Работа с Docker. Часть 2.

Создадим собственные образа Docker.

Используйте машину, подготовленную с помощью Vagrant.

Этап 1. Создание образа из контейнера.

Давайте сначала создадим и запустим контейнер из образа Ubuntu (ubuntu).

```
docker container run -it --name my-ubuntu Ubuntu
```

Нажмите комбинацию Ctrl+P и Ctrl+Q, чтобы закрыть соединение, не закрывая контейнер.

```
docker container ls -f name=my-ubuntu
```

Нажмите комбинацию Ctrl+P и Ctrl+Q, чтобы закрыть соединение, не закрывая контейнер.

Мы можем проверить, работает ли контейнер, но на этот раз применим фильтр:

```
docker container ls -f name=my-ubuntu
```

Процесс создания образа из контейнера (даже работающего) очень прост:

```
docker container commit --author "RUT-USER" my-ubuntu new-ubuntu
```

Давайте сначала проверим, работает ли контейнер.

```
docker container ls -f name=my-ubuntu
```

А потом проверяем, есть ли наш новый образ

```
docker image ls new-ubuntu
```

Давайте проверим, можем ли мы создать и запустить контейнер на его основе.

На этот раз мы запустим контейнер с одной дополнительной опцией, которая заставит контейнер автоматически удаляться после остановки.

```
docker container run -it --rm new-ubuntu
```

Да, можем, и это работает. Теперь закройте сеанс

```
root@fc0bc2b9b8ab:/# exit
```

Если мы проверим список остановленных контейнеров, мы не найдем его там, так как использовали параметр --rm

```
docker container ls -a
```

Этап 2. Создание образа с помощью heredoc.



Облачные платформы и виртуализация

Heredoc — синтаксис занесения в переменную одно- или многострочного свободно форматированного текста «как есть».

Это метод создания простых многострочных текстовых документов на лету.

Его также можно использовать для создания простых образов Docker, создавая Dockerfile на лету.

Давайте выполним следующую команду

```
docker image build -t alp-htop - << EOF  
FROM alpine  
RUN apk --no-cache add htop  
EOF
```

У нас появился новый образ в списке?

```
docker image ls
```

Теперь мы можем запустить контейнер на основе нашего нового образа.

```
docker container run -it alp-htop
```

Проверяем, что мы можем использовать добавленный пакет (**alp-htop**)

```
/ # htop
```

А потом закрыть сессию

```
/ # exit
```

Этап 3. Создание образа из Dockerfile.

Мы создадим два одинаковых с точки зрения пользователя образа, используя файл Dockerfile.

Создадим папку nginx-1 и перейдем в нее

```
mkdir nginx-1 && cd nginx-1
```

Теперь создайте пустой Dockerfile для редактирования (используйте nano, если хотите)

```
nano Dockerfile
```

Введите следующий текст

```
FROM ubuntu  
LABEL maintainer="RUT-USER"  
RUN apt-get update  
RUN apt-get install -y nginx
```



Облачные платформы и виртуализация

```
ENTRYPOINT ["/usr/sbin/nginx","-g","daemon off;"]
```

EXPOSE 80

Сохраните и закройте файл

Следующим шагом является генерация или сборка образа.

Сборка образа выполняется с помощью

```
docker image build -t nginx-1 .
```

После создания образа разверните контейнер на его основе с помощью

```
docker container run -d -p 8080:80 --name web-1 nginx-1
```

Пробросьте порт 8080 в виртуальной машине и откройте веб-адрес <http://localhost:8080>

У вас должен открыться веб-сайт Nginx

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working.
Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Мы можем остановить контейнер с помощью следующей команды

```
docker container stop web-1
```

Теперь перейдите в верхнюю папку

```
cd ..
```

Создайте еще одну новую папку и перейдите в нее

```
mkdir nginx-2 && cd nginx-2
```

Откройте новый пустой файл для редактирования

```
nano Dockerfile
```

Введите следующий текст

```
FROM ubuntu
```

```
LABEL maintainer="RUT-USER"
```

```
RUN apt-get update && apt-get install -y nginx
```

```
ENTRYPOINT ["/usr/sbin/nginx","-g","daemon off;"]
```



Облачные платформы и виртуализация

EXPOSE 80

Сохраните и закройте файл

Создайте новый образ с помощью

```
docker image build -t nginx-2 .
```

Запустите контейнер из нового образа

```
docker container run -d -p 8080:80 --name web-2 nginx-2
```

Если мы снова на хосте откроем вкладку браузера и посетим <http://localhost:8080>, мы увидим тот же результат.

Теперь остановите контейнер с помощью следующей команды

```
docker container stop web-2
```

Итак, для нас результат один и тот же, а для Docker это не так.

Давайте рассмотрим первый образ с помощью

```
docker image history nginx-1
```

Вывод должен быть похож на

IMAGE COMMENT	CREATED	CREATED BY	SIZE
b16421e0b876	3 minutes ago	/bin/sh -c #(nop) EXPOSE 80	0B
3d5c566eb9a8	3 minutes ago	/bin/sh -c #(nop) ENTRYPOINT ["/usr/sbin/ng...	0B
96a3afaaa16e	3 minutes ago	/bin/sh -c apt-get install -y nginx	59.2MB
a96d134e620d	4 minutes ago	/bin/sh -c apt-get update	33.7MB
92309843847a	4 minutes ago	/bin/sh -c #(nop) LABEL maintainer= RUT-US ...	0B
54c9d81cbb44	7 days ago	/bin/sh -c #(nop) CMD ["bash"]	0B
<missing>	7 days ago	/bin/sh -c #(nop) ADD file:3ccf747d646089ed7...	72.8MB

А потом второй

```
docker image history nginx-2
```

Вывод должен быть похож на

IMAGE COMMENT	CREATED	CREATED BY	SIZE
335de904496b	About a minute ago	/bin/sh -c #(nop) EXPOSE 80	0B
ff29f1a69e9c	About a minute ago	/bin/sh -c #(nop) ENTRYPOINT ["/usr/sbin/ng...	0B
ec1107f8aad4	About a minute ago	/bin/sh -c apt-get update && apt-get install...	92.8MB
92309843847a	4 minutes ago	/bin/sh -c #(nop) LABEL maintainer=RUT-USE ...	0B
54c9d81cbb44	7 days ago	/bin/sh -c #(nop) CMD ["bash"]	0B
<missing>	7 days ago	/bin/sh -c #(nop) ADD file:3ccf747d646089ed7...	72.8MB



Облачные платформы и виртуализация

Как видно, оба образа имеют разное количество слоев (семь против шести).

Разница заключается в разном количестве команд RUN в двух файлах Dockerfile.

В любом случае, когда мы проверяем их размер

```
docker image ls
```

Он тот же самый.

Теперь перейдите в верхнюю папку

```
cd ..
```

Создайте еще одну новую папку и перейдите в нее

```
mkdir nginx-3 && cd nginx-3
```

Откройте новый пустой файл для редактирования

```
nano Dockerfile
```

Введите следующий текст

```
FROM ubuntu
```

```
LABEL maintainer="RUT-USER"
```

```
RUN apt-get update && apt-get install -y nginx && rm -rf  
/var/lib/apt/lists/*
```

```
ENTRYPOINT ["/usr/sbin/nginx","-g","daemon off;"]
```

```
EXPOSE 80
```

Сохраните и закройте файл

Создайте новый образ с помощью

```
docker image build -t nginx-3 .
```

Запустите контейнер из нового образа

```
docker container run -d -p 8080:80 --name web-3 nginx-3
```

Если мы снова на хосте откроем вкладку браузера и посетим <http://localhost:8080>, мы увидим тот же результат.

Теперь остановите контейнер с помощью следующей команды

```
docker container stop web-3
```

Опять же для нас результат один и тот же а для Docker это не так

```
docker image history nginx-3
```



Облачные платформы и виртуализация

Вывод должен быть похож на

IMAGE COMMENT	CREATED	CREATED BY	SIZE
f9e661a365e7	2 minutes ago	/bin/sh -c #(nop) EXPOSE 80	0B
b85ce1089c69	2 minutes ago	/bin/sh -c #(nop) ENTRYPOINT ["/usr/sbin/ng...	0B
0b5861ca838f	2 minutes ago	/bin/sh -c apt-get update && apt-get install...	92.8MB
92309843847a	19 minutes ago	/bin/sh -c #(nop) LABEL maintainer=RUT-USE ...	0B
54c9d81cbb44	7 days ago	/bin/sh -c #(nop) CMD ["bash"]	0B
<missing>	7 days ago	/bin/sh -c #(nop) ADD file:3ccf747d646089ed7...	72.8MB

Снова шесть слоев, как в попытке №2.

Давайте еще раз проверим размер трех образов.

```
docker image ls nginx-*
```

Хорошо, нам удалось сэкономить 30+ МБ 😊